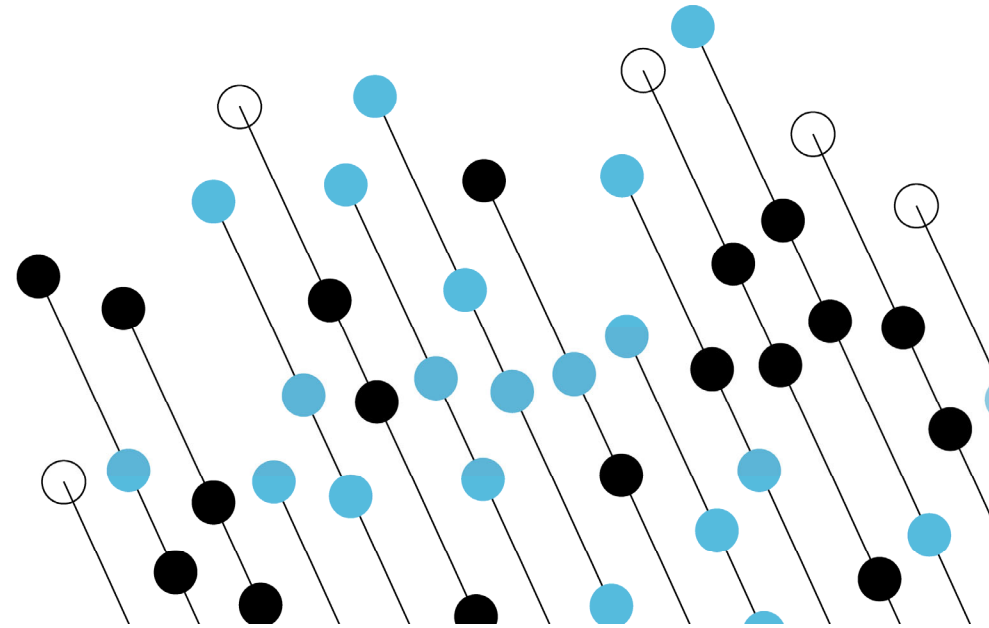


Static fields and methods



Contents



Objectives

- To understand types and how to create objects.
- To grasp the concept of reference type behaviour.



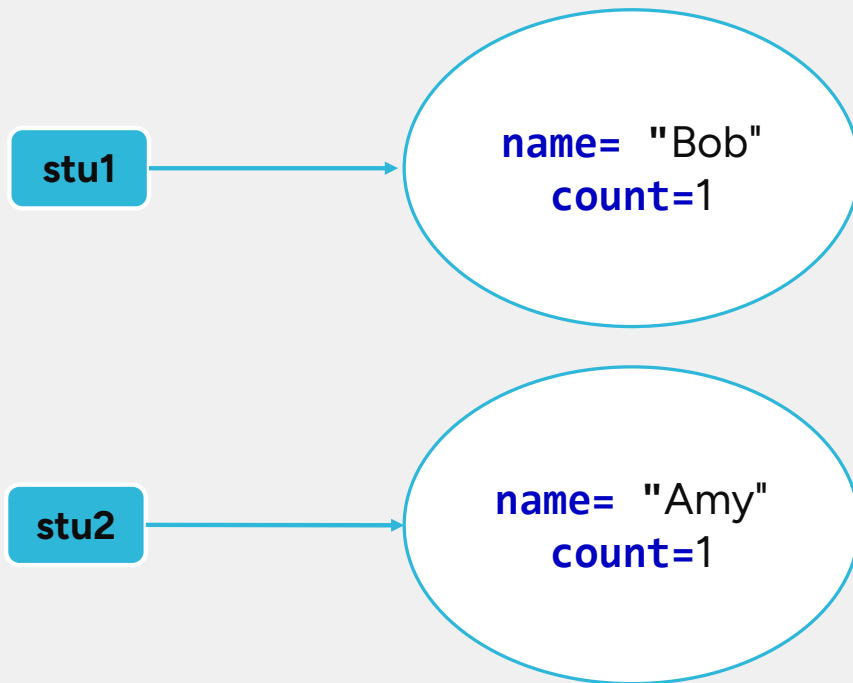
Contents

- OO fundamentals — Abstraction and encapsulation
- Defining reference types
- Creating objects (instances)



Hands-on labs

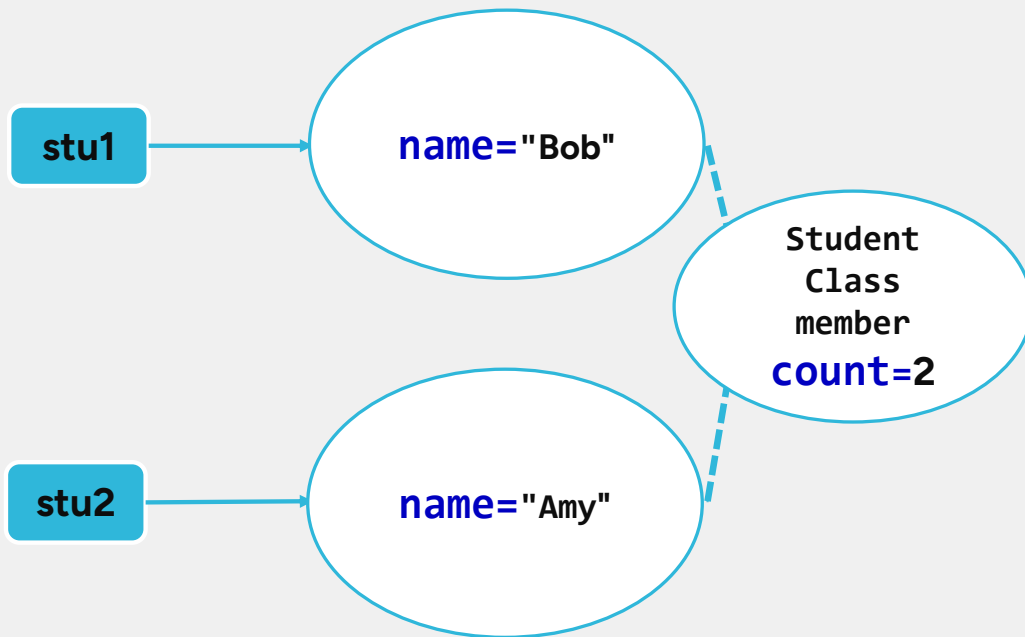
Objects in memory



```
class Student {  
    public int count = 0;  
    private String name;  
  
    public Student(String name) {  
        this.name = name;  
        this.count++;  
    }  
}
```

```
Student stu1 = new Student("Bob");  
Student stu2 = new Student("Amy");
```

Static members



```
class Student {  
    ➡ public static int count = 0;  
    private String name;  
  
    public Student(String name) {  
        this.name = name;  
        this.count++;  
    }  
}
```

```
Student stu1 = new Student("Bob");  
Student stu2 = new Student("Amy");
```

```
int total = Student.count;
```

Static — a few framework classes with static methods

Static means 'belongs to the class, not to an instance of the class'.

Static members visible via the **class name**.

There's no need to create an instant.

```
System.out.println("OK");
```

```
System.out.println(Math.PI);
```



```
System s = new System();
```

```
s.out.println();
```



```
Math m = new Math();
```

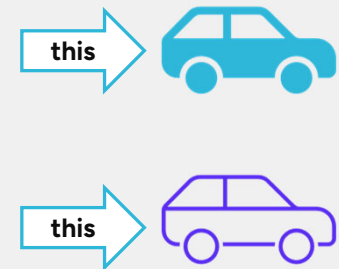
```
m.sqrt(25.6);
```



this

- **this** refers to the object on which method was invoked.
 - You could write accelerate method as follows:

```
public class Car {  
    private int speed;  
    public void accelerate( int weightOfFoot ) {  
        this.speed += weightOfFoot;  
    }  
    ...  
}
```



Using static — simple factory pattern

Package
A

```
public class Bank {  
    public static Account createAccount(string owner) {  
        // possible security code  
        return new Account(owner);  
    }  
}
```

Factory class

```
public class Account {  
    Account(string owner) {  
    }  
}
```

Constructor is only visible
inside this package

Package
B

```
public static void main(String[] args)  
{  
    Account acc = new Account("Bob"); ✗  
  
    Account acc = Bank.createAccount("Bob"); ✓  
}
```

Client code using the
bank account factory in
another package

More on static members

- Can access other static members
- Cannot access instance member
 - Can only access by using an object reference
- Instance methods can call static methods and use static fields

```
public class Car {  
    private String make, owner;  
  
    public Car(String make) {  
        this.make = make;  
    }  
  
    public static Car makeBMW() {  
        Car aCar = new Car("BMW");  
        aCar.setOwner("Tom");  
        return aCar;  
    }  
  
    public void setOwner(String owner) {  
        this.owner = owner;  
    }  
    ...  
}
```


Java static initialiser block

- **Cannot mark a constructor as static, but can define static block:**
 - Runs when the class is first accessed.
 - Executes before any method calls or object creation.
 - Used to initialise complex static fields.

```
public class Car {  
    static licenseAgency DVLA;
```

```
    static {  
        if(...) DVLA = new LicenseAgency( ... );  
    }
```

```
    public static void setLicenseAgency (LicenseAgency la) {  
        DVLA = la;  
    }
```

```
}
```

final fields

- **final** variables can be assigned to in declaration or in a constructor.
 - Then they are read-only — they can be instance or static members.

```
public class Car {  
    public final int yearOfCreation;  
  
    public Car() {  
        yearOfCreation = Calendar.getInstance().get(Calendar.YEAR);  
    }  
}
```

```
public class Math {  
    public static final double PI = 3.14159265358979323846;  
}
```

Creating global constants to avoid magic values

```
class AppConfig
{
    public static final String APP_NAME = "My Java game";
    public static final int    MAX_USERS = 100;
}

public class Main
{
    public static void main(String[] args)
    {
        System.out.println("Welcome to " + AppConfig.APP_NAME);
        System.out.println("Max users allowed: " + AppConfig.MAX_USERS);
    }
}
```

Names in caps

Review

Type members are either:

- Static (belong to type itself).
 - Always accessed via class name.
 - No instance needed to use them.
- Instance (belong to an instance of the type).
 - Accessed via instance reference.

```
Math.pow(2,3);
```

```
mycar.accelerate(10);
```



Lab

Use static members:

- We need a Vehicle Count.
- Each Vehicle needs a Registration Plate.
- Create a Registration Plate Factory to provide the next plate.

